

## SPPL-195: Apply Generic Shipping Pattern Post API technical workflow

Jira Ticket: [📌 SPPL-195: BE | Create technical workflow for Shipping Pattern Get & Post APIs on parameter inputs page including DB details](#) ACCEPTED

 This page provides comprehensive information about the POST API for Apply Generic Shipping Pattern on the Input params page, to generate & apply shipping dates based on user provided weekly pattern.

**Apply Generic Shipping Patterns**

Populate shipping patterns for dates beyond the uploaded coverage.

Uploaded coverage (Last vanning date): **21 Jul 2025**

Apply To

- ☐ Entire plan duration
- ☒ All months outside uploaded coverage

Select Vessel Loading Day

| Mon                 | Tue                 | Wed                 | Thu                 | Fri                 | Sat                 | Sun                 | Mon                 | Tue                 | Wed                 | Thu                 | Fri                 | Sat                 | Sun                 |
|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|
| Vanning day (Fixed) |                     |                     |                     | Vessel load day     |                     |                     |                     |                     |                     |                     |                     |                     |                     |
|                     | Vanning day (Fixed) |                     |                     |                     |                     | Vessel load day     |                     |                     |                     |                     |                     |                     |                     |
|                     |                     | Vanning day (Fixed) |                     |                     |                     |                     | Vanning day (Fixed) |                     |                     |                     |                     |                     |                     |
|                     |                     |                     | Vanning day (Fixed) |                     |                     |                     |                     | Vanning day (Fixed) |                     |                     |                     |                     |                     |
|                     |                     |                     |                     | Vanning day (Fixed) |                     |                     |                     |                     | Vanning day (Fixed) |                     |                     |                     |                     |
|                     |                     |                     |                     |                     | Vanning day (Fixed) |                     |                     |                     |                     | Vanning day (Fixed) |                     |                     |                     |
|                     |                     |                     |                     |                     |                     | Vanning day (Fixed) |                     |                     |                     |                     | Vanning day (Fixed) |                     |                     |
|                     |                     |                     |                     |                     |                     |                     | Vanning day (Fixed) |                     |                     |                     |                     | Vanning day (Fixed) |                     |
|                     |                     |                     |                     |                     |                     |                     |                     | Vanning day (Fixed) |                     |                     |                     |                     | Vanning day (Fixed) |
|                     |                     |                     |                     |                     |                     |                     |                     |                     | Vanning day (Fixed) |                     |                     |                     | Vanning day (Fixed) |
|                     |                     |                     |                     |                     |                     |                     |                     |                     |                     | Vanning day (Fixed) |                     |                     | Vanning day (Fixed) |
|                     |                     |                     |                     |                     |                     |                     |                     |                     |                     |                     | Vanning day (Fixed) |                     | Vanning day (Fixed) |
|                     |                     |                     |                     |                     |                     |                     |                     |                     |                     |                     |                     | Vanning day (Fixed) | Vanning day (Fixed) |

Vessel load to NAMC arrival days

- 21
- 22
- 23
- 18
- 21
- 22
- 20

Months affected: Mar 2025- Mar 2028

Reset to previous version

1049 Hug × 662.28 Hug

Cancel Recalculate

### Supply Planning POST Generic Shipping Pattern API Details:

1. **Hosted URL:** <https://api.spln.cube><env>.toyota.com/<env>
2. **API Name:** <HOSTED\_URL>/v/1/1/sp/apply-generic-shipping-pattern
3. **Path Param:** N/A
4. **Query Param:** N/A
5. **Method:** POST
6. **Authorization:** Bearer token (**required**)
7. **Lambda Name:** spln-<env>-apply-generic-shipping-pattern-v1-1
8. **API Request Body:**

```
1 {
2   "scenarioId": "uniqueScenarioId",
3   "entirePlanDuration": true,
4   "userEmail": "user@toyota.com",
5   "vanningCenter": "RAV4",
6   "data": [
7     {
8       "vanningDay": 0,
9       "loadDay": 4,
```

```

10     "loadToArrivalDays": 14,
11   },
12   {
13     "vanningDay": 1,
14     "loadDay": 7,
15     "loadToArrivalDays": 20,
16   },
17   {
18     "vanningDay": 10,
19     "loadDay": 13,
20     "loadToArrivalDays": 24,
21   },
22 ]
23 }

```

## 9. API Response:

- **Success response with status 200:**

```

1 {
2   "message": "Successfully updated data."
3 }

```

- **Error Response:**

- **any internal server error occurred with status 500:**

```

1 {
2   "errorMessage": "Internal Server Error"
3 }

```

- **any unauthorized error occurred with status 401 (Managed by the AWS API Gateway integrated Custom Authorizer):**

```

1 {
2   "message": "Unauthorized"
3 }

```

- **If request is in-valid with status 400:**

```

1 {
2   "errorMessage": [<"ValidationError: validation error message">]
3 }

```

- **List of possible validation error message:**

1. "ValidationError: Request body cannot be empty."
2. "ValidationError: scenarioId is required and must be a uuid."
3. "ValidationError: entirePlanDuration must be either true or false."
4. "ValidationError: userEmail is required and must be a string."
5. "ValidationError: vanningCenter is required and must be a string."
6. "ValidationError: Scenario doesn't exist."
7. "ValidationError: Invalid userEmail."
8. "ValidationError: data is required and must be an array with atleast 1 item."

9. "ValidationError: vanningDay is required and must be a number between 0 to 6."
10. "ValidationError: loadDay is required and must be a number between 0 to 13."
11. "ValidationError: loadToArrivalDays is required and must be a non-negative number not more than 99."
12. "ValidationError: Incomplete data provided."
13. "ValidationError: No TMC working day calendar data found for scenarioId <scenarioId> and vanningCenter <vanningCenter>"
14. "ValidationError: Shipping pattern data already exists for the entire scenario duration. No update was performed."
15. "ValidationError: vanningDay must include all days 0-6. Vanning Day <> is missing."

#### 10. Tabl

- a. **scenarios**
- b. **tmc\_working\_day\_calendar**
- c. **generic\_pattern**
- d. **shipping\_pattern**
- e. **scenario\_step\_status**

#### SP Apply Generic Shipping Pattern Post API Workflow:

##### • Technical workflow:

- Description:
  - i. The Supply Planning user logs in to the application.
  - ii. After the user provides creates a scenario, opens it and applies Generic Shipping Pattern, a request is sent to the SP Post Generic Shipping Pattern endpoint: **sp/apply-generic-shipping-pattern** with request body:

```
1 {
2   "scenarioId": "uniqueScenarioId",
3   "entirePlanDuration": true,
4   "userEmail": "user@toyota.com",
5   "vanningCenter": "RAV4",
6   "data": [
7     {
8       "vanningDay": 0,
9       "loadDay": 4,
10      "loadToArrivalDays": 14,
11    },
12    {
13      "vanningDay": 1,
14      "loadDay": 7,
15      "loadToArrivalDays": 20,
16    },
17  ]
18 }
```

```

17     {
18         "vanningDay": 10,
19         "loadDay": 13,
20         "loadToArrivalDays": 24,
21     },
22 ]
23 }

```

- iii. After successful authentication, the **spln-apply-generic-shipping-pattern-v1-1** lambda is triggered by the SP scenario apply generic Shipping Pattern endpoint: **sp/apply-generic-shipping-pattern**. If authentication fails, an error with status 401 (refer: error response-**any unauthorized error occurred with status 401**) will be returned and sent to client.
- iv. After user authentication has been confirmed, the **spln-apply-generic-shipping-pattern-v1-1** lambda will perform validation on the request payload. If payload is valid will proceed to next step else throw validation error (refer: error response: **Section if request is in-valid with status 400 and List of possible validation error message**). Below queries are used to check if provided scenario is valid & vanning dates are working days:

```

1 | select * from supply_planning.scenarios where scenario_id = :scenarioId and
  | is_active = true;

```

- v. Check if the user has provided loadDay for all days vanningDays else throw validation error (11).
- vi. After the request payload is successfully validated, the **sp-apply-generic-shipping-pattern-v1-1** lambda execute logic to generate shipping pattern based on the inputs provided for either entire timeframe or partial timeframe (remaining timeframe for which data doesn't exist in file).
- vii. Fetch TMC Working Day Calendar data for given scenarioId and vanningCenter in order to identify working & non-working days using below query:

```

1 | select tmc_date          as "prodDate",
2 | day_of_week             as "dayOfWeek",
3 | is_working_day          as "isWorkingDay",
4 | work_day_percentage     as "workDayPercentage",
5 | week_number             as "week"
6 | from supply_planning.tmc_working_day_calendar
7 | where scenario_id = ${body.scenarioId}::uuid
8 | and vanning_center = ${body.vanningCenter}
9 | order by tmc_date;

```

- viii. In case of no TMC Working Day Calendar data for given scenarioId and vanningCenter, throw validation error (13) as atleast prefilled data must exist in the **tmc\_working\_day\_calendar** table.

- ix. If we have partial timeframe request and the complete timeframe shipping pattern data already exists in the **shipping\_pattern** table, throw validation error (14).
- x. Below query to post scenario Generic Shipping Pattern data into the **generic\_pattern** & **shipping\_pattern** table.

```
1  -- if user selects partial timeframe, fetch existing data from
   shipping_pattern table
2  -- to identify remaining timeframe else use scenario timeframe from query
   response (iv)
3  SELECT * from supply_planning.shipping_pattern where scenario_id =
   :scenarioId and is_active = true;
4  -- Upsert generic_pattern table data
5  INSERT INTO supply_planning.generic_pattern (
6      scenario_id,
7      vanning_day,
8      vessel_load_day,
9      vessel_to_namc_days,
10     apply_to_all,
11     vanning_center,
12     created_by,
13     created_date_timestamp
14 )
15 SELECT
16     ${scenarioId}::uuid AS scenario_id,
17     v.vanning_day,
18     v.vessel_load_day,
19     v.vessel_to_namc_days,
20     v.apply_to_all,
21     ${vanningCenter}::text AS vanning_center,
22     ${userEmail}::text AS created_by,
23     NOW() AS created_date_timestamp
24 FROM (
25     VALUES
26     ${Prisma.join(
27         data.map(
28             (item) => Prisma.sql`
29                 ${item.vanningDay}::integer,
30                 ${item.loadDay}::integer,
31                 ${item.loadToArrivalDays}::integer,
32                 TRUE::boolean
33             `
34         )
35     )}
36 ) AS v(vanning_day, vessel_load_day, vessel_to_namc_days,
   apply_to_all)
37 ON CONFLICT (scenario_id, vanning_center, vanning_day)
38 DO UPDATE SET
39     vessel_load_day = EXCLUDED.vessel_load_day,
40     vessel_to_namc_days = EXCLUDED.vessel_to_namc_days,
41     apply_to_all = EXCLUDED.apply_to_all,
42     updated_by = ${userEmail}::text,
43     last_updated_timestamp = NOW()
44
45 -- Upsert shipping_pattern table data
46 await prisma.$executeRaw`
47     INSERT INTO supply_planning.shipping_pattern (
48         scenario_id,
49         namc,
50         vanning_center,
51         vanning_date,
52         vanning_day,
53         tmc_working,
54         carrier,
```

```

55         vessel_load_date,
56         namc_arrival_date,
57         created_by
58     )
59     SELECT
60         ${scenarioId}::uuid AS scenario_id,
61         v.namc,
62         v.vanning_center,
63         v.vanning_date,
64         v.vanning_day,
65         v.tmc_working,
66         v.carrier,
67         v.vessel_load_date,
68         v.namc_arrival_date,
69         ${userEmail}::text AS created_by
70     FROM (
71         VALUES
72         ${Prisma.join(
73             rows.map(
74                 (item) => Prisma.sql`
75                 ${item.namc}::text,
76                 ${item.vanningCenter}::text,
77                 ${item.vanningDate}::date,
78                 ${item.vanningDay}::text,
79                 ${item.isWorkingDay}::boolean,
80                 ${item.carrier}::text,
81                 ${item.vesselLoadDate}::date,
82                 ${item.namcArrivalDate}::date
83             )`
84         )
85     )}
86     ) AS v(
87         namc,
88         vanning_center,
89         vanning_date,
90         vanning_day,
91         tmc_working,
92         carrier,
93         vessel_load_date,
94         namc_arrival_date
95     )
96     ON CONFLICT (scenario_id, vanning_center, vanning_date)
97     DO UPDATE SET
98         tmc_working = EXCLUDED.tmc_working,
99         vessel_load_date = EXCLUDED.vessel_load_date,
100        namc_arrival_date = EXCLUDED.namc_arrival_date,
101        updated_by = ${userEmail}::text,
102        last_updated_timestamp = NOW()
103 `;

```

- xi. Update the step & scenario status to “In Progress”. Only update scenario status to “In Progress” if its not already using scenarioData from query (iv) response.

```

1  insert into supply_planning.scenario_step_status (scenario_id,
2  input_step_type, input_step_name, status, created_by, created_date_timestamp)
3  values (${scenarioId}::uuid, ${stepType}, ${stepName}, ${status},
4  ${userEmail}, now())
5  on conflict (scenario_id, input_step_name)
6  do update set status = ${status}, updated_by = ${userEmail},
7  last_updated_timestamp = now()
8  returning scenario_step_status_id::uuid;
9
10 UPDATE supply_planning.scenarios
11 SET scenario_status = ${scenarioStatus},
12     updated_by = ${userEmail},

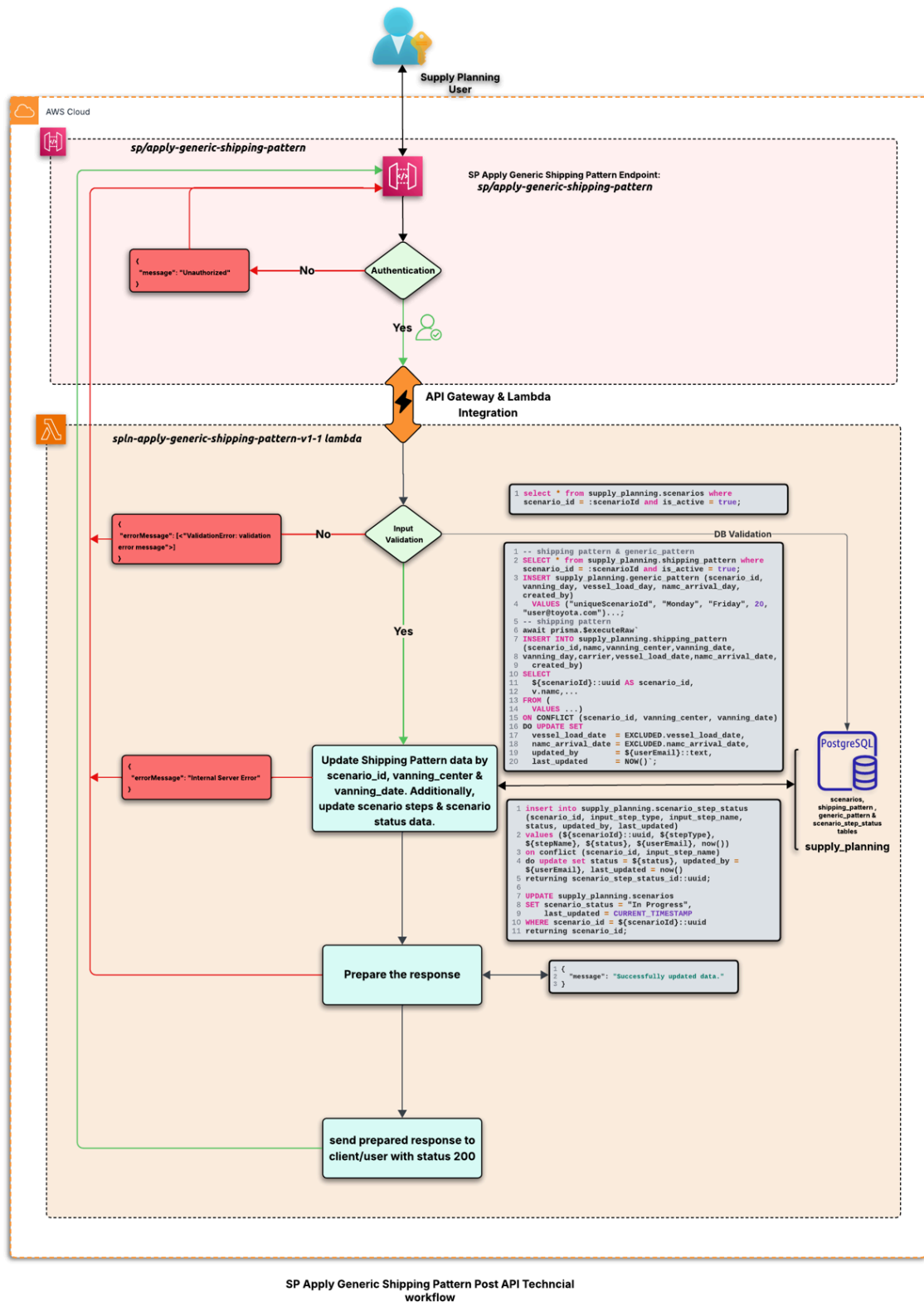
```

```
10     last_updated_timestamp = CURRENT_TIMESTAMP
11 WHERE scenario_id = ${scenarioId}::uuid
12 returning scenario_id;
```

- xii. After updating the Shipping Pattern data, return the below response to the client with a status code of 200:

```
1  {
2    "message": "Successfully updated data."
3  }
```

- xiii. If there is an error while updating the data, an “Internal Server Error” with a status code of 500(*refer: error response-**any internal server error occurred with status 500***) will be returned. Otherwise, a successful response with a status code of 200 will be sent.



SP Apply Generic Shipping Pattern Post API Technical workflow

## Note:

We will be storing both working & non-working days to shipping\_pattern table. Non-working days will have vessel\_load\_date & namc\_arrival\_date as null.





**Queries:**

What will the carrier be for the generic pattern data? “-”

Do we shipping pattern data for all vaning centers by the provided generic pattern when clicked on Recalculate? Based on the filter they have selected

While calculating namc arrival date count both working days & non-working days.